

Crash Course i Numme SF1550

Disa Degerholm

29/05-2026

Kapitel.....	1
F1-2 Felanalys	1
F3-5 Olinjär/iterativ ekvationslösning (ej $f(ax+by)=af(x)+b(y)$)	1
F5-7 Linjära algebra: ekvationssystem och egenvärden	3
F8-9 Interpolation & minstakvadratmetoden	4
F9-11 Numerisk derivering & integrering	5
F12 Optimering	6
Kod	7
Matte	8

Kapitel

F1-2 Felanalys

- Exakta fel

$x = \tilde{x} + \varepsilon_x \iff \text{värde} = \text{approximation} + \text{indatafel}$.

Där y definieras på liknande sätt, alt (om $F \in C^1$) $\varepsilon_y \approx \varepsilon_x F'(\tilde{x})$.

- Felgränser ger maxfel

- X: felgränser F: Absolut $F = E_x \geq |\varepsilon_x|$ och relativ $F = R_x \geq \left| \frac{\varepsilon_x}{x} \right|$.
- Y: (om $F \in C^1$) $E_y \approx E_x |F'(\tilde{x})|$ kallas fortplantningsformeln och $R_y \approx \frac{E_x |F'(\tilde{x})|}{|y|}$.
- Konditionerat = Väl om $|F'(\tilde{x})|$ är litet och dåligt om motsats. Mäts i $R_y \approx \kappa R_x$.

- Värdesiffror = korrekta siffror / signifikanta siffror, fås av $0.5 * 10^{g-v} > \varepsilon_y$ där g = grundpotensform av talet, v = antal korrekta värdesiffror, d = antal korrekta decimaler. Tex $13,790 \pm 0,005$ $v=4$, $g=1$, $d=3$ ty $d=v-g$

- Korrekta decimaler (d) = fås av $0.5 * 10^{-d} > \varepsilon_y$

- Ex: $123,456 \pm 0,005$ $v=4$, $g=2$, $d=2$ ty sista decimalen är osäker och $v=2+2$ enklast se men kan också tänka $+0,005$ påverkar 2 decimaler
- Ex: $123,0 \pm 0,5$ $v=2$, $g=2$, $d=0$ observera att $+/-$ ändrar även 3:an
- Ex: $123,456 \pm 0,009$ $v=3$, $g=2$, $d=1$ (härled d med formeln ovan och testa d)

- Trunkeringsfel (numme - ändra x_0 eller h) = approximationsfel/diskretiseringsfel är fel från metod

- Indatafel (indata - störningsanalys/fortplantning/konditionstal) = modellfel/parameterfel är förenklignars osäkerheter i parametrar/modell (tex mätfel, fel modell). Störningsanalys $E_y \approx F(\tilde{x} + E_x)$, används då felfortplantning inte kan hitta derivata. Konditionstal = mäter medfödd känslighet oavsett metod ($Ax=b$), störningsanalys = mäter känslighet givet viss fel i input (allt), felfortplantning = mäter känslighet givet viss fel i input (om $F(x_n) \in C^n$).

- Avrundningsfel (dator - skriv om uttryck) = maskinfel från hur datorns representerar tal

- Utskiftning vs Kancelation = Båda beror på avrundningsfel men skillnaden är $+/-$ av tal i mycket olika storleksordning (dator kan ej lagra alla d så den tar bara dom som matchar), den andra är subtraktion mellan två näsan lika tal (datorn lagrar redan tidiga tal och när de blir noll finns bara svans kvar med få v).
- Extra = Jämförelse med floats är inte alltid exakt lika

F3-5 Olinjär/iterativ ekvationslösning (ej $f(ax+by)=af(x)+b(y)$)

- Metoder: Obs om multipel rot finns dvs $f^m(x^*) = 0$ blir ko långsammare

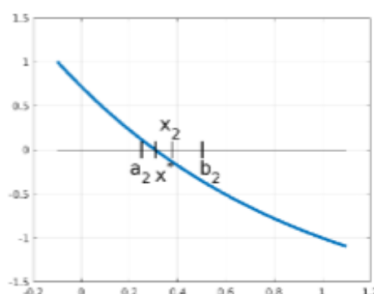
- Namn: Intervallhalvering (ej fixP)

- * Anta: f skalär kontinuerlig, x^* ligger i $[a_0, b_0]$, $f(a_0)f(b_0) < 0$

- * Metod: $x_0 = \frac{a_0+b_0}{2}$, om $f(x_0)f(a_0) < 0$ så $b_0 = x_0$ annars $a_0 = x_0$ och kalla det $[a_1, b_1]$ osv tills $|b_n - a_n|$ är tillräckligt litet.

- * KO ordning: 1, $S=0.5$, $e_n \leq (b_0 - a_0)2^{-n+1}$ ty $n = 0, \dots, n$.

- * Pro/Con: +klarar brett intervall, +ingen f' krävs, +konvergerar alltid, -fungerar bara skalär, -är långsam

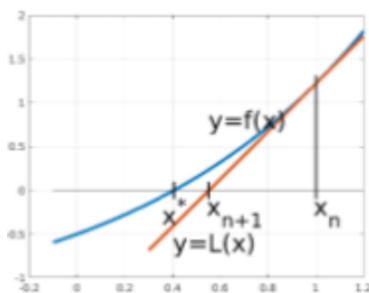


– Namn: Fixpunktsiteration

- * Anta: nära startgissning
- * Metod: fixpunktsform $f(x)=x$, nollställeform $g(x)=f(x)-x=0$
- * KO ordning: ofta 1, ju mindre derivata och delvis S desto snabbare. $|g'(x^*)| < 1$ ger lokal linjär konvergens (>1 di i allmänhet, $=1$ mer info behövs). Ko ordning: $\frac{|e_{n+1}|}{|e_n|^p} \approx S$ om $p=1$ $S = |g'(x^*)| + \mathcal{O}(e_n)$ (få genom $g(x^* + e_n)$ taylor i x^*) där $0 < |S| < 1$ (mellan -1 och 1 men ej 0) givet x^* är tillräckligt nära rot. Där p = konvergensordning och S = asymptotisk konvergensthastighet. Obs om $p > 1$ (superlinjär) gäller $0 < |S| < \infty$ ty e_n^p kommer dominera vs konstanten S medan för $p=1$ så förändras bara e_n av S .
 - **Sats startintervall:** om $g \in C^1$, $|g'(x)| < 1 \forall x$ i hela symmetriska intervallet runt fixpunkten (tex $|x| < 2,5$ då $x^*=1 \rightarrow [-0.5, 2.5]$) då ko fixpunktsiterationen för alla startgissningar i det intervallet (övriga punkter är osäkra).
 - **Sats lokal ko:** om $g \in C^1$, $|g'(x^*)| < 1$ ko fixpunktsmetoden om startgissningen är tillräckligt nära.
 - **Sats global ko:** om $g \in C^1$, $|g'(x)| < 1 \forall x \in \mathbb{R} \rightarrow$ ko fixpunktsmetoden för alla startgissningar (det finns alltid en fixpunkt då) $\rightarrow x^*$ är unik om finns. Obs om derivatan max ko mot $p < 1$ så finns det garanterat en fixpunkt.
- * Pro/Con: +Går alltid skapa ej unik tex $f(x)=g(x)+x$, +fungerar för ekvsys hastighet varierar, -kan ge falska rötter -ko inte alltid, -kräver bra startgissningar

– Namn: Newtons metod

- * Anta: finns minst ett nollställe, nära startgissning, $f \in C^2$, $f'(x^*) \neq 0$
- * Metod: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$ för skalär.
- * KO ordning: ofta 2 men 1 om x^* är en dubbelrot/högre multiplicitet dvs $f(x^*) = f'(x^*) = 0$
- * Pro/Con: +alltid lokalt ko ty $g'(x^*)=0$ vilket gör att område runt x^* finns s.a <1 , +ko snabbt, +fungerar för ekvsys, -derivata krävs, -kräver bra startgissningar



– Namn: Sekantmetoden (ej fix)

- * Anta: 2 startgissningar
- * Metod: Newton men derivatan mellan x_n, x_{n-1} dvs $x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$
- * KO ordning: under 2
 - **Sats lokal ko sekant:** om $f \in C^2$, $f'(x^*) \neq 0$ och startgissningarna är tillräckligt nära så ko sekantmetoden med gyllene snittet ty $e_{n+1} \approx \frac{f''(x^*)}{2f'(x^*)} e_n e_{n-1}$ som fås av att taylorutveckla def av sekantmetoden.
- * Pro/Con: +kräver ej derivata, +lokalt ko för enkla rötter (ej multipel), +ko hastighet knappt 2, -kräver 2 bra startgissningar, -svårt för system

– Namn: Modifierade Newtons metod

- * Anta: NaN
- * Metod: Newton men derivatan approximeras med framåt differans dvs $x_{n+1} = x_n - \frac{hf(x_n)}{f(x_n+h)-f(x_n)}$
- * KO ordning: knappt 2 ty $g'(x^*) = O(h)$
- * Pro/Con: +kräver ej derivata, +lokalt ko för små h, +ko nästan 2, +fungerar för system, -kräver 2 evalueringar/iteration, -känslig för val av h pga avrundningsfel (litet h->kancelation m.m pga dators precision, stort h->trunkeringsfel), -kräver bra startgissning
- Hitta startgissningar: förenkla problem med matten ($\sin(x)=x$ då $x=0.05$, $e^x=0$ då $e \ll 1$ och $x \pm 1$, vid $\max f'_x = f'_y = 0$, eller samband som ges), förenkla fysiken (ett rimligt svar är ca, gör enklare version och använd det svaret), gör plot och hitta skärning för 0-linjer, skalär (intervallhalvering)
- Feluppskattning då x^* saknas: för linjär ko $|S-1||e_{n-1}| \approx |x_n - x_{n-1}|$ där $(1-S)|x_n - x^*| \approx |S||x_n - x_{n-1}|$ (sista formeln fungerar för $p=2$ men överskattar felet). Om kvadratisk -> ca fördubblas korrekta decimaler/iteration $|x_n - x^*| \approx 10^{-d} \rightarrow |x_{n+1} - x^*| \approx S * 10^{-2d}$ och vi har att $e_{n-1} \approx |x_n - x_{n-1}|$. Genrellt används $|x_{n+1} - x_n| \approx S|x_n - x_{n-1}|^p$.

F5-7 Linjära algebra: ekvationssystem och egenvärden

- Beräkningskostnad (flops): beräkningstid $\approx \frac{\text{flops}}{\text{flops/s}} \iff$ om samma metod används $(n_0)^p$ som tog s sekunder förut och n_0 blir n_1 fås $(n_1/n_0)^p * s$ = ökning av tid = där $*s$ = nya tiden. Alternativt kan man räkna ut $\text{flops/s} = \text{komplexitet}(\text{storlek}_1)/t_1$ och sedan $\text{flops/s} * t = \text{komplexitet}(\text{storlek}_2)$ för att lösa ut t =nya tiden.
 - Viktiga formler: $v^* M a_{ij} := \sum_k b_{ik} c_{kj}$ kramas å dim måste stämma
 - $v^T v O(n)$
 - $Mv O(n^2)$
 - $MM O(n^3)$
 - fullt $Ax = b \frac{2}{3}n^3 = O(n^3)$
 - triangulärt $Ax = b O(n^2)$
 - bandat $Ax = b/Mv/MM O(n)$
 - LU förberedelser $O(n^3)$, $O(n^2)$ för lösa efter. Algoritmen: $A = LU$ där L är nedre triangulärt och U är övre triangulärt, lös $Ly = b$ och sedan $Ux = y$. Obs gör pivotering (byte av rader s.a att största elementet i kolumnen är på diagonalen) för att minska avrundningsfel.
- Normer vektorer: $\|x\|_2 := \sqrt{\sum_{j=1}^n x_j^2}$, $\|x\|_1 := \sum_{j=1}^n |x_j|$, $\|x\|_\infty := \max_j |x_j|$ (np.linalg.norm(x, np.inf)).
- Normer matriser: $\|A\| := \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}$ som ger $\|A\|\|x\| \geq \|Ax\|$. Mer specifikt $\|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$ (max summa av rad där element är i belopp), $\|A\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$ (max kolumn summa med värden i belopp), $\|A\|_2 = \max_j \sqrt{|\lambda_j(A^T A)|}$ (där det inom absolutbeloppet är egenvärden av $A^T A$).
- Konditionstal: $\kappa(A) = \|A\| \|A^{-1}\| = \frac{|\lambda_{\max}|}{|\lambda_{\min}|}$ (felförstärkning) där relativa felet i utdata begränsas av relativa felet i indata (används mest för matriser). Vi har också $R_x \leq \kappa(A) R_y$, dvs stort κ = nästan singulär matris/illa konditionerad små fel i x ger stora fel i y. OBS! y=indata (det man valt som mål) och x=utdata (det man behöver hitta som lösning). Kod np.linalg.cond(A, norm).
- Metoder ekvssys
 - Funktions sys: $F(x)=0$, $J(x)$ =(kolonner är x_i och raderna är f_i). Linjärisering i x_0 ger $L(x) = F(x_0) + J(x_0)(x - x_0)$ obs om bara flervarje blir derivatan ∇ annars f'. Ko om $0 < \rho(J(x^*)) < 1 \iff 0 < \max_j |\lambda_j| < 1$ och ko linjärt samt snabbare ju mindre ρ (spektralradien) är. Derivatan går från: $\rho(J(x^*)) \rightarrow H(x^*)$ (Hessian) $\rightarrow T(x^*)$ (tredjegradstendorn).
 - Newton: $x_{n+1} = x_n - \delta$ där $J(x_n)\delta = F(x_n)$. Samma krav som skalär lokalt ko om $F \in C^2$ alltid, kvadratisk ko om $J(x^*)$ är icke singulär snabbt, -kräver alla partiella derivator, -kräver bra startgissning.

- Modifierade Newton: samma men $J(x) \approx \begin{bmatrix} \frac{f(x+h,y)-f(x,y)}{h} & \frac{f(x,y+h)-f(x,y)}{h} \\ \frac{g(x+h,y)-g(x,y)}{h} & \frac{g(x,y+h)-g(x,y)}{h} \end{bmatrix}$
- Matris sys: $A=P-B \Leftrightarrow x = P^{-1}(Bx+b) \Leftrightarrow Px_{n+1} = Bx_n + b$ å skriv om till $x_{n+1} = \dots$ där P är en approximation av A som är lätt att invertera (B följer). Extra: ko om $0 < \rho(P^{-1}B) < 1$ och snabbare ju mindre ρ är. Ty $\rho(A) \leq \|A\| \forall$ matrisnormer fås förenklingen att metoden ko om $\|P^{-1}B\| < 1$.
 - * **Sats Jacobi/Seidel ko:** Jacobi å gauss-Seidel ko $\forall A \in R^{N \times N}$ om $|a_{kk}| > \sum_{l \neq k, k=1}^N |a_{kl}|$ dvs A är diagonaldominant (diagonalelement radvis i belopp är större än summan av radens värden absolut individuellt (utan diagonalelementet)). Extra: Seidel ko också för symmetriska positivt definita A .
- Jacobis metod: $P = \text{diag}(A)$, löses sen som vanligt.
- Gauss-Seidels metod: $P = \text{undertriangulära}(A)$, löses mha föregående rads lösning osv nedåt $\rightarrow$ ofta snabbare konvergens än Jacobi.
- Metoder egenvärden
 - Viktiga egenskaper: om A är reel symmetrisk blir alla λ reella och egenvektorer ortogonala. Om A är $n \times n$ och har n st λ, v så är den diagonaliserbar. $(v_i, \lambda_i) = \text{egenmod}$. Största egenvektor/värde absolut är dominant.
 - Potensmetoden (fix)
 - * Metod: $v_{k+1} = Ay_k, \lambda_k = y_k v_{k+1}, y_{k+1} = \frac{v_{k+1}}{\|v_{k+1}\|}$ kan tolkas som: startgissning av egenvektor kommer närmare den riktiga, sen genom skalärprodukten mäts ca skalfaktorn dvs egenvärdet, sen normeras nuvarande egenvektor gissning för att iterera.
 - * Pro/Con: +bred startgissning funkar
 - * KO: Låt $A \in R^{n \times n}$ va diagonaliserbar (funkar utan men långsammare) och ha $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ upp till multiplicitet $\rightarrow$ ko $\lambda_k \rightarrow \lambda_1$ för nästan alla y_0 , dvs ej $y_0 \in \text{span}\{v_2, \dots, v_n\}$ sällan problem pga avrundningsfel. $\lambda_k = \lambda_1 + O(S^k), y_k = \pm v + O(S^k)$. $S = \frac{|\lambda_2|}{|\lambda_1|}$ (snabbare om symmetrisk $S = |\lambda_2/\lambda_1|^2$). Obs om λ_1 inte är reelt och positivt ko kanske inte v till rätt tecken $g(v) = \frac{Av}{\|Av\|} = \frac{\lambda}{|\lambda|}v$ medan λ_1 fortfarande ko till rätt värde ty $\lambda_k = y_k^T Ay_k$.
 - * Extra: Inversa hittar minsta egenvärdet, skift hittar det som är närmast skiftet.

F8-9 Interpolation & minstakvadratmetoden

- Interpolation (exakt: oscilerar för att nå max antal punkter)
 - Existens: Det finns ett unikt polynom av gradtal $\leq n$ som går genom de punkterna $n+1$ givet att $n+1$ punkter finns och är distinkta. Av högre gradtal finns oändligt många.
 - Ansatser (obs blir samma $p(x)$ i slutändan)
 - * Naiv: $p(x) = c_0 + c_1x + \dots + x_n x^n$ som blir linjärt ekvationsystem för alla par (x, y) kallas Vandermonde-matrisen (inverterbar, illa konditionerad stora n).
 - * Newton: $p(x) = d_0 + d_1(x-x_0) + d_2(x-x_0)(x-x_1) + \dots + d_n \prod_{j=0}^{n-1} (x-x_j)$ vilket för de olika (x, y) paren ger undertriangulärt system (bra konditionerat, lättare lösa).
 - * Lagrange: $p(x) = \sum_{j=0}^n L_j(x)f(x_j)$ där $L_j(x) = \prod_{k=0, k \neq j}^n \frac{x-x_k}{x_j-x_k}$ där $j=0, \dots, n$.
 - Maximala interpolationsfel: på intervall för polynom av grad n med $n+1$ ekvidistanta punkter gäller $E_n = \max_{x \in [a,b]} |p_n(x) - f(x)| \leq \max_{x \in [a,b]} \frac{|f^{n+1}(x)|}{(n+1)!} h^{n+1}$ där $e_n = f(x) - p(x) = \frac{f^{n+1}(\xi)}{(n+1)!} (x-x_0)(x-x_1)\dots(x-x_n)$.
 - Runges fenomen: då $|f^{n+1}(x)|$ växer snabbare än övriga faktorer avtar tex i E_n leder det till att kanter divergerar, dvs att polynomet divergerar/oscilerar våldsamt i ändar då $n \rightarrow \infty$. TLDR: ekvidistant interpolation med högt gradtal är dåligt.
 - Styckvis linjär polynominterpolation: ko då $n \rightarrow \infty$ ty är begränsade till linje på liten del där $E_n \leq Ch^2$. Men för generellt gradtal d med $d+1$ punkter gäller $E_n \leq Ch^{d+1}$. OBS att styckvislinjär

interpolation är $O(h^2)$ innebär att E_y är mindre eller lika med det $\forall x \in [\text{interpolerade intervallet}]$.
Hittas mellan två punkter $[x_1, y_1]$ och $[b_1, h_1]$ genom $y = y_0 + k(x - x_0)$ dvs $y=kx+m$ bara skiftat till nytt origo.

- Splines: styckvis högre interpolation. Om splinen ska va av grad n så ska alla f^{n-1} vara kontinuerliga.
- Linjära minstakvadratmetoden (generell: oscilerar för att få minsta totala residual)
 - Normal ekvationen: $A^T A x = A^T y$ ($A^T A$ kallas normal matris) räkna för hand samt dimensioner. Där A ofta är vandermondmatrisen. Extra: $\kappa(A^T A) = \kappa(A)^2$ riskabel.
 - Residualvektorn: $r = F(x) = Ax - y$, norm som vill minimeras $\|r\|_2$, felkvadratsumma $\|r\|_2^2$.
 - Kod: `np.linalg.lstsq(A, b)`
- Linjära ersättningsmodeller: logaritmera icke linjära exponentiella modeller tex $y = c_0 e^{c_1 x} \Leftrightarrow \ln(y) = \ln(c_0) + c_1 x \Leftrightarrow Y = C_0 + c_1 x \rightarrow c_0 = e^{C_0}$. Andra ex: $y = c_0 x^{c_1}$ (ok transformera x). OBS nu förändras vad som minimeras med dina nya variabler.
- Ickelinjära minstakvadratmetoden Gauss Newton (som Newton för system men $r(x) = F(x) \approx 0$ med MKM istället för = och gausselimination)
 - Gauss Newton givet modell/data: given icke linjärmodell och datapunkter kan man skriva om sambandet till en residual $F(x) = r(x) \approx 0$ som man vill minimera dvs $\|F(x)\|_2$ mha $x_{n+1} = x_n - \delta$ där $J(x_n)\delta \approx F(x_n)$. $p \leq 2$, är fixpunktsiteration.
 - Gauss Newton överbestämda antal krav: saknas mätdata och modellfunktion men finns överbestämt ekvationsystem av diverse krav tex från fysikaliskt problem som mha minstakvadratmetoden kan lösas.

F9-11 Numerisk derivering & integrering

- Numerisk derivering/differenskvoter (taylor av f' def, obs behöver ofta grad+2 för härleda)
 - Framåtdifferens: $f'(x) \approx \frac{f(x+h)-f(x)}{h}$ (vanlig f'). $O(h)$
 - Bakåtdifferens: $f'(x) \approx \frac{f(x)-f(x-h)}{h}$ (jämför med punkt bak, lik sekant med där h =helt steg). $O(h)$
 - Central-differens: $f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$ (mean av fram och bak differens). $O(h^2)$
- Numerisk integrering/kvadraturmetoder
 - Generellt: $\int_a^b f(x)dx \approx \sum_{j=0}^n w_j f(x_j)$ där w är vikter och x noder.
 - Trapetregeln: exakt integrering av styckvis linjär interpolation. $I_h(f) = \frac{h}{2}(f(x_0) + 2 \sum_{j=1}^{n-1} f(x_j) + f(x_n)) = h(\sum_{j=0}^n f(x_j) - \frac{f(x_0)}{2} - \frac{f(x_n)}{2})$ sista är lättare för kod. Felet $E_h \leq Ch^2$ som kommer från att $E_n \leq \int_a^b |f(x) - p_h(x)|dx \leq (b-a) \max_{x \in [a,b]} |f(x) - p_h(x)|$ dvs är baserat på interpolationsfelet $\rightarrow p=2$. 2D rektanglar: lös en integral i taget mha diskretisering av $n+1$ och $m+1$ punkter som blir

$$I \approx \sum_{j=0}^n \sum_{k=0}^m f(x_j, y_k) h_x h_y w_{j,k} \text{ där } w_{j,k} = \begin{cases} 1, \text{ inre punkt} \\ 1/2, \text{ kant punkt} \\ 1/4, \text{ hörn punkt} \end{cases}, p=2. \text{ Generellt metod med } p \text{ i}$$
 dim d blir $O(N^{-p/d})$. För att gälla måste f va snäll C^2 .
 - Simpsons formel: exakt integration av styckvis kvadratisk interpolation, $p=4$.
 - Monte-Carlo: $\int_{\Omega} f(x)dx \approx \frac{|\Omega|}{N} \sum_{j=1}^N f(X_j)$ där det finns N st slumpmässigt utplacerade punkter i volymen/arean Ω och är baserat på formeln för väntevärdet. Felet är nästan standardavvikelse $O(N^{-1/2})$ dvs oberoende av dimension. Kod TODO ksk titta
 - Precisionsgrad: d = fås av en enkel regel om den är exakt $\forall$ monomen $1, x, \dots, x^d$ men ej $x^{d+1}, \dots$. Enkel regel kan sättas till vadsomhelst om inget specificeras men ansats kan göras tex $c_0 f(0) + c_1 f(y) c_2 f(1)$ och sedan hitta koefcenter med utveckling till sammansatta/enkla vad man har att jämföra med - som sedan kan användas för att hitta precisionsgrad osv. (Newton-Cotes formler = enkla regler med ekvidistanta punkter). hitta sammansatt regel med generellt intervall som skrivs om till enkla regeln

(vars sub var är $+$ var vill komma* h generellt) som summeras till stora intervallet (summan ta enkla ex för att hitta mönster), $p_sammansatt = d_enkla + 1$ (för enkel, interpolerande, regel med d om $f \in C^p$). Högre ordning Newton-Cotes med n ekvidistanta punkter gäller $d = \begin{cases} n-1, n \text{ jämnt} \\ n, n \text{ udda} \end{cases}$

och $p = \begin{cases} n, n \text{ jämnt} \\ n+1, n \text{ udda} \end{cases}$. Höga n riskerar Runge's fenomen och kanter kan då tas bort "öppna regler" istället för slutna regler".

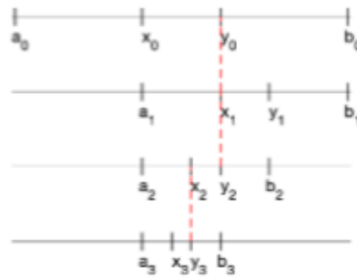
- Singulära/Generaliserade integraler: ∞ intervall (lös upp till L och uppskatta absolut storlek på svans genom förenkling, alternativt sub för ändra intervall), integrand i intervall (sub , singularitetsubtraktion $\pm g(x)$ som ko men gör integrand gränsvärde ko), integrands $\neq C^p$ för metoden (sub).

- Nogrannhetsordning: p plottas ofta i log-log-plot Ch^p vs $h \rightarrow p = \frac{\log(y_1) - \log(y_0)}{\log(x_1) - \log(x_0)}$. I en sådan plott finns: avrundningsfel då h är litet $\frac{f(x+h)+e_1-f(x)+e_2}{h} \approx f'(x) + Ch + \frac{e_1+e_2}{h}$, asymptotiska konvergensområdet då förväntat beteende finns, stora h så börjar nästa högre ordningens term i felet dominera. Dvs krav för att $E_n \approx Ch^p$ är u_h är en snäll funktion ej diskontinuerlig i f eller f' etc (annars $E_n \leq Ch^p$), h är tillräckligt litet s.a högre ordningens termer kan försummas. Givet detta gäller $e_{h/2} \approx \frac{u_h - u_{h/2}}{2^p - 1}$. Vill man få en övre gräns kan man ta $E_{h/2} = |e_{h/2}| \leq |u_h - u_{h/2}|$. Detta betyder ca att de siffror som inte förändras i steglängdshalveringen kan tolkas korrekta. Utan minst 3 u_{h_i} vet vi inget om p genom $\frac{u_h - u_{h/2}}{u_{h/2} - u_{h/4}} \approx 2^p$.

- Reguljär ko : ger $e_h \approx Ch^p$ (antas om ej annat sägs).
- Felskattningar: $|e_{h/2}| \approx |u_h - u_{h/2}|$. Mer avancerad version: $e_{h/2} \approx \frac{u_h - u_{h/2}}{2^p - 1}$ intuitionen (hur mycket fel finns kvar till rätta svaret när halverar h).
- Richardson-extrapolation (öka $p \rightarrow +1$): Genom p elimineras ledande felet genom $R_{h/2} = u_{h/2} - \frac{u_h - u_{h/2}}{2^p - 1}$ (ide: ta bästa värde hitills och ta bort samma nogrannhets fel $u_{h/2} - e_{h/2} \approx u^*$ alternativt $R_{h/2} = u_{h/2} - e_{h/2}$ som ungefär blir närmare det sanna svaret). $R(u_trapetz) = u_simpson$ $p: 2 \rightarrow 4$ mer än $+1$!

F12 Optimering

- Konvex sats punkter: om f är kontinuerlig och området kompakt (slutet & begränsat) $\rightarrow$ det existerar globala max och min punkter (vi letar dock lokal minimum, maximum är $-f$). Om x är lokal extrempunkt är den antingen en kritisk punkt $\nabla f(x) = 0$ (topp av kulle), randpunkt (kant), singulär punkt $\nabla f(x)$ existerar ej (spets).
- Konvex sats punkt: konvex är då man inte kan rita rakt sträck från två punkter i figuren och hamna utanför. Strikt konvex funktion är då man drar rakt linje från två punkter i grafen och linjen är över grafen i det området. När $f \in C^2(D)$ är f konvex om $f'' \geq 0$ 1D eller Hessianen H är positivt semidefinit $+D$. När f är strikt konvex och D är konvext samt kompakt $\rightarrow$ har f ett entydigt globalt minimum (lokala är då globala).
- Unimodal: då f har ett unikt minimum $x^* \in [a_0, b_0]$ och f är strikt avtagande för $x < x^*$ och strikt växande för $x > x^*$. Dvs ser ut som en ledsen mun.
- Metoder
 - 1 dimension
 - * Förenklad gyllendesnitt: Originell inspiration. Välj x_0, y_0 s.a $a_0 < x_0 < y_0 < b_0$. Om $f(x_0) > f(y_0)$ byt till $[x_0, b_0]$ annars $[a_0, y_0]$ (välj nya kant till lägsta värde) - kalla nya intervallet $[a_1, b_1]$. Upprepa tills $|b_n - a_n| < tol$.
 - * Gyllendesnittet Sökning: Återvänd tidigare funktionsvärden (bara ett nytt krävs/iter) för att minska beräkningskostnaden. Välj x_0, y_0 s.a $a_0 < x_0 < y_0 < b_0$ och s.a $\frac{y-a}{b-a} = \frac{b-x}{b-a} = \frac{\sqrt{5}-1}{2} \approx 1,62$ dvs minskar med den faktorn/iteration. Om $f(x_0) > f(y_0)$ byt till $[x_0, b_0]$ annars $[a_0, y_0]$, bryt vid villkor. Om $b - a = L_0$ blir $L_{n+1} = q * L_n = q^{n+1} * L_0$. Och de inre punkterna för varje iteration blir $x_2 = a + (1 - q)L$ samt $x_3 = a + qL$.



- * Newtons metod opt: samma som newton fast allt en derivata högre ty hittar då $f'(x)=0$.
- +1 dimensioner
- * Newtons metod opt: $x_{k+1} = x_k - H(x_k)^{-1} \nabla F(x_k)$, beräkningskrävande ofta och bra startgissningar.
- * Gauss Newton opt: $x_{k+1} = x_k - (J(x_k)^T J(x_k))^{-1} J(x_k)^T f(x_k)$ används som approximation till Newton när $A \neq n \times n$ utan $A = m \times n$ där $m \gg n$ dvs överbestämt och i detta fall för optimering. Ex senaste tenta.
- * Gradientsökning: $x_{k+1} = x_k - \alpha_k \nabla F(x_k)$ där α väljs eller längs dess riktning $\alpha_k = \operatorname{argmin}_{\alpha} F(x_k - \alpha \nabla F(x_k))$ som hittas med andra numeriska metoder.

Kod

Numpy = Matte

SciPy = Numme

Matplotlib = Plotta

Kodvana

- Struktur
 1. Matteformler: funktioner, jacobianer, metoden
 2. bibliotek
 3. förberedande: startgissning, konstanter, funktion/derivata, ibland placeholder kommande värden, tol, delta - ca 5 saker
 4. metod i loop
 5. printa
- använd $f(x)=x^2+1$ i loopar eller utanför om det endast används en gång tex `np.sum(f)`
- använd `def funk(x):` // return indenterat, om uttrycket ska användas med många olika värden utanför en loop
- `np.array([rad, vektor]) ([[kolumn], [vektor]]) ([a, b], [c, d])`
- `np.linspace([start, stop, n+1])` minnesregel rymden har små punkter som heter stjärnor
- `np.arange([start, stop+1, h])`, utan h blir i steglängd 1, minnesregel du ska exakt hit på denna bredd
- Vanliga: `np.abs()`, `np.sum()` kräver f och array alt `u[1:-1]`, `np.linalg.norm()`, `np.zeros(r, k)`, `np.eye(n)`, `.T`, `np.ones(r, k)`
- `np.linalg.solve(A, b)`, OBS $f(x)$ om funktion
- `d, _, _, _ = np.linalg.lstsq(J(c), g(c))`
- `np.column_stack((k1, ..., kn))` OBS är oftare lättare skriva ut för hand i matris array
- $M^*v = M @ v$ eller `MM` eller `vTv`. `vvT` är `np.outer(a, b)`

- Loopar
 - while condition True: ...
 - for i in range (start, stopp-h, h): ... vanligt med matriser J[idx, :]=uttryck
- Extra
 - np.reshape(array, rad, kol), om -1 i ett arg så hittar den matchande baserat på resten
 - uttryck; uttryck ... kan skriva mer på en rad alternativt a, b, c = 1, 2, 3
- Antaganden
 - Om kod fortsätts senare skriv Fortsätter från ovan"
 - Om startgissning inte behöver hittas skriv x = Sätt in startgissning"

Matte

Derivata trigonometri: $\sin \Rightarrow \cos \Rightarrow -\sin \Rightarrow -\cos$

I frågor på prov behövs det inte skrivas variabler, formler eller frågan i sig som redan är skriven i lydelsen om man själv inte vill för att vara lättläslig.

Logaritmlagar

- $\log(a * b) = \log(a) + \log(b)$
- $\log(a/b) = \log(a) - \log(b)$